BAB 1

Interupsi BIOS/DOS dan Operasi I/O

Outcome Pembelajaran Khusus Bab (Sub-CPMK)

Sub-CPMK 2.4 — Menjelaskan mekanisme interupsi dan layanan BIOS/DOS (`INT 10h`, `INT 21h`) sebagai jembatan antara instruksi mesin dan perangkat keras serta operasi masukan/keluaran.

CPMK yang didukung: CPMK02 • **CPL yang didukung: CPL02**

Tujuan Pembelajaran

Setelah mempelajari bab ini, mahasiswa mampu:

1. menjelaskan alasan diperlukannya interupsi dan membedakan interupsi perangkat keras, perangkat lunak, serta pengecualian prosesor;
2. menghitung alamat vektor interupsi ke-n dan menyebutkan vektor-vektor penting beserta kegunaannya;
3. menelusuri urutan penyimpanan `FLAGS`, `CS`, dan `IP` pada stack serta pengambilannya kembali oleh `IRET`;
4. menggunakan layanan BIOS `INT 10h`, `INT 16h`, `INT 17h` serta layanan DOS `INT 21h`;
5. memasang *interrupt handler* sederhana dengan mengganti isi sebuah vektor, lalu memulihkan vektor tersebut.

Materi Pokok

1. Konsep interupsi: perangkat keras, perangkat lunak, dan pengecualian.
2. Tabel vektor interupsi pada alamat `0000h`–`03FFh` dan cara menghitung alamat vektor ke-n.
3. Mekanisme `INT` dan `IRET` serta kaitannya dengan `TF` dan eksekusi satu langkah.
4. Layanan BIOS: `INT 10h`, `INT 16h`, dan `INT 17h`.
5. Layanan DOS `INT 21h` beserta penyangga fungsi `0Ah` dan perbedaan `INT 20h` dengan fungsi `4Ch`.

6. Penanganan interupsi buatan sendiri beserta risikonya, serta studi kasus program sapaan dan teks berwarna.

1.1 Konsep Interupsi dan Tabel Vektor

Pada Bab **??**, layanan `INT 21h` dipakai sebagai kotak hitam. Bab ini membuka kotak hitam tersebut: pertanyaannya bukan lagi "bagaimana memakai" layanan, melainkan "apa yang sesungguhnya terjadi" ketika `INT` dijalankan.

1.1.1 Mengapa Interupsi Diperlukan

Prosesor berjalan jauh lebih cepat daripada perangkat di sekitarnya. Bila program terus-menerus memeriksa status keyboard, sebagian besar waktunya terbuang untuk pengulangan yang hampir selalu sia-sia. Cara ini disebut *polling*.

Definisi 1.1. Interupsi adalah mekanisme yang memaksa prosesor menghentikan sementara alur program, memindahkan kendali ke rutin khusus yang disebut **penangan interupsi** (*interrupt handler*), lalu melanjutkan kembali program semula tepat pada instruksi berikutnya.

Interupsi juga menjadi sarana baku untuk meminta layanan sistem: program cukup memanggil `INT 10h` tanpa menulis sendiri rutin pengendali video [**?**].

1.1.2 Tiga Jenis Interupsi

Berdasarkan sumber pemicunya, interupsi dibagi menjadi tiga jenis.

1. **Interupsi perangkat keras** berasal dari luar prosesor melalui kaki `INTR` atau `NMI`. Interupsi melalui `INTR` bersifat *maskable*: dapat ditahan bila `IF` bernilai nol. Interupsi melalui `NMI` tidak dapat ditolak dan dipakai untuk kejadian gawat.
2. **Interupsi perangkat lunak** dipicu instruksi `INT n` di dalam program. Instruksi ini sengaja dijalankan untuk meminta layanan; seluruh layanan BIOS dan DOS termasuk jenis ini.
3. **Pengecualian** (*exception*) dibangkitkan prosesor ketika terjadi keadaan yang tidak diinginkan. Instruksi `DIV` dengan pembagi nol memicu vektor `00h`, sedangkan `INTO` memicu vektor `04h` bila `OF` bernilai satu.

Perbedaan ketiga jenis tersebut dirangkum pada Tabel 1.1.

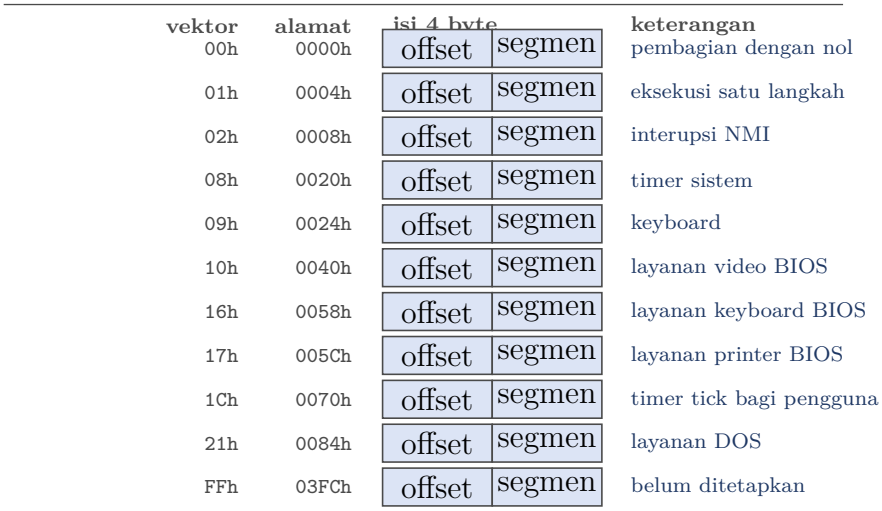
Tabel 1.1: Tiga jenis interupsi pada prosesor 8086.

Jenis	Sumber	Contoh	Sifat
Perangkat keras	Kaki `INTR` atau `NMI`	Timer 18,2 kali per detik (`08h`); penekanan tombol (`09h`)	Terjadi kapan saja, tidak direncanakan program
Perangkat lunak	Instruksi `INT n`	`INT 21h` fungsi `09h` untuk mencetak string	Disengaja, hasilnya dapat diperkirakan
Pengecualian	Prosesor, akibat kesalahan eksekusi	Pembagian dengan nol (`00h`); limpahan `INTO` (`04h`)	Menandakan kesalahan yang harus ditangani

1.1.3 Tabel Vektor Interupsi

Prosesor harus mengetahui letak rutin penangan setiap interupsi. Untuk itu, 256 kemungkinan interupsi pada 8086 diberi nomor `00h` sampai `FFh`, dan alamat setiap penangan disimpan pada **tabel vektor interupsi** (*interrupt vector table*). Tabel ini menempati 1.024 byte pertama memori, yaitu alamat `0000h` sampai `03FFh` pada segmen nol, dan mampu menampung $1.024 \div 4 = 256$ vektor.

Setiap entri terdiri atas 2 byte **offset** penangan yang dimuat ke `IP`, diikuti 2 byte **segmen** yang dimuat ke `CS`. Bagian bernilai lebih rendah diletakkan pada alamat yang lebih rendah. Tata letaknya tampak pada Gambar 1.1.



Gambar 1.1: Tata letak tabel vektor interupsi pada segmen nol, dari alamat `0000h` sampai `03FFh`. Entri berisi alamat penangan, bukan penangan itu sendiri.

1.1.4 Menghitung Alamat Vektor ke-n

Karena setiap vektor berukuran tetap, alamat awal vektor bernomor n dihitung dengan rumus berikut:

$$\text{alamat vektor}_n = n \times 4 \tag{1.1}$$

Hasilnya adalah offset di dalam segmen nol. Vektor `09h` (keyboard) berada pada $9 \times 4 = 36 = \texttt{24h}$, vektor `10h` (video BIOS) pada $16 \times 4 = 64 = \texttt{40h}$, dan vektor `21h` (layanan DOS) pada $33 \times 4 = 132 = \texttt{84h}$. Vektor yang sering dijumpai pada praktikum dirangkum pada Tabel 1.2.

Tabel 1.2: Vektor interupsi penting beserta alamat dan kegunaannya.

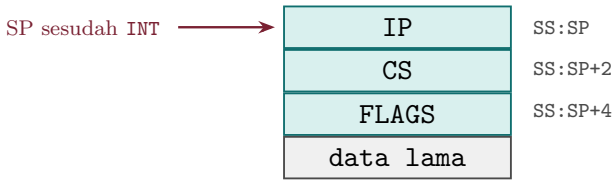
Vektor	Alamat entri	Pemicu	Kegunaan
`00h`	`0000h`	Pengecualian prosesor	Pembagian dengan nol, dipicu otomatis oleh `DIV`
`01h`	`0004h`	Pengecualian prosesor	Eksekusi satu langkah, dipicu saat `TF` bernilai satu
`08h`	`0020h`	Perangkat keras	Timer sistem, dibangkitkan 18,2 kali per detik
`09h`	`0024h`	Perangkat keras	Penekanan dan pelepasan tombol keyboard
`10h`	`0040h`	Perangkat lunak	Layanan video BIOS: mode layar, kursor, penulisan karakter
`16h`	`0058h`	Perangkat lunak	Layanan keyboard BIOS: membaca dan memeriksa tombol
`17h`	`005Ch`	Perangkat lunak	Layanan printer BIOS
`1Ah`	`0068h`	Perangkat lunak	Waktu dan tanggal dari jam perangkat keras
`1Ch`	`0070h`	Perangkat lunak	Timer *tick* bagi pengguna, dipanggil dari penangan `08h`
`21h`	`0084h`	Perangkat lunak	Layanan DOS: masukan, keluaran, berkas, terminasi program

1.1.5 Mekanisme `INT` dan `IRET`

Instruksi `INT n` bekerja seperti pemanggilan prosedur jarak jauh, tetapi dengan tambahan penyimpanan *flag* [**?**]. Urutannya:

1. isi register `FLAGS` disimpan ke stack;
2. register `CS` disimpan ke stack;
3. register `IP` — yang ketika itu sudah menunjuk instruksi sesudah `INT n` — disimpan ke stack;
4. `IF` dan `TF` dikosongkan, sehingga interupsi perangkat keras ditahan sementara dan eksekusi satu langkah tidak aktif;
5. alamat penangan dibaca dari entri vektor pada alamat $n \times 4$; dua byte pertama dimuat ke `IP` dan dua byte berikutnya ke `CS`.

Ketiga penyimpanan itu menurunkan `SP` sebanyak 6 byte. Isi stack sesudahnya tampak pada Gambar 1.2.



SP berkurang 6 byte. Pengambilan kembali oleh `IRET` dilakukan dalam urutan terbalik.

Gambar 1.2: Isi stack sesudah instruksi `INT n` dijalankan, dengan `SP` menunjuk pada nilai `IP` yang tersimpan paling akhir.

Instruksi `IRET` melakukan tiga pengambilan berurutan, yaitu `IP`, `CS`, lalu `FLAGS`, persis kebalikan urutan penyimpanan. Bandingkan dengan `CALL` pada Bab ?? yang hanya menyimpan `CS` dan `IP`: `IRET` setara dengan `RET` ditambah `POPF`.

Kesalahan yang Sering Terjadi

Rutin penangan interupsi wajib diakhiri `IRET`, bukan `RET`. Bila `RET` dipakai, dua byte teratas stack yang diambil sebagai alamat kembali justru berisi `FLAGS`, sehingga prosesor melompat ke alamat acak dan program menghentikan diri atau sistem membeku.

1.1.6 Flag `TF` dan Eksekusi Satu Langkah

Bit ke-8 register `FLAGS` adalah **trap flag** `TF`. Bila bit ini bernilai satu, prosesor menjalankan satu instruksi lalu segera membangkitkan interupsi vektor `01h`. Mekanisme inilah yang dipakai *debugger* untuk menyediakan tombol `Single Step` pada Emu8086 atau perintah `T` pada DEBUG [?].

Peran `TF` menjelaskan langkah keempat mekanisme `INT` di atas: bila flag itu tetap bernilai satu, penangan akan memicu vektor `01h` setiap kali satu instruksinya selesai

sehingga prosesor terjebak. Pemakaiannya pada penelusuran program dibahas pada Bab **??**.

1.2 Interupsi BIOS: Video, Keyboard, dan Printer

1.2.1 Kedudukan BIOS

BIOS (*Basic Input/Output System*) adalah kumpulan rutin yang tersimpan pada cip memori baca-saja dan dipetakan ke alamat `F000h` pada ruang memori. Rutin ini berbicara langsung dengan pengendali video, pengendali keyboard, dan port printer, lalu menyajikan antarmuka yang seragam bagi program di atasnya [**?**]. Karena perbedaan perangkat keras disembunyikan di sini, program tidak perlu mengenal nomor port perangkat.

> **Catatan**
> Seluruh layanan BIOS memakai konvensi yang sama: `AH` berisi nomor fungsi, register lain berisi argumen, dan hasil dikembalikan pada register.

1.2.2 `INT 10h`: Layanan Video

Interupsi `10h` melayani keperluan tampilan. Fungsi yang paling sering dipakai pada praktikum dirangkum pada Tabel 1.3.

Tabel 1.3: Fungsi `INT 10h` yang dipakai pada bab ini.

Fungsi	Register masukan	Register keluaran	Keterangan
`00h`	`AL` = nomor mode video	—	Menetapkan mode; `03h` berarti teks 80 × 25 berwarna, dan layar dibersihkan
`02h`	`BH` = halaman; `DH` = baris; `DL` = kolom	—	Memindahkan posisi kursor tanpa mencetak apa pun
`03h`	`BH` = halaman	`DH`, `DL` = posisi kursor; `CX` = bentuk kursor	Membaca posisi kursor, berguna untuk menyimpan lalu memulihkan posisi
`06h`	`AL` = jumlah baris gulung; `BH` = atribut; `CH`, `CL`, `DH`, `DL` = batas jendela	—	Menggulung jendela ke atas; `AL` = `00h` menggulung seluruh jendela
`09h`	`AL` = karakter; `BH` = halaman; `BL` = atribut; `CX` = jumlah penulisan	—	Menulis karakter berwarna berulang; kursor *tidak* bergeser
`0Ah`	`AL` = karakter; `BH` = halaman; `CX` = jumlah penulisan	—	Menulis karakter memakai atribut yang sudah ada di layar; kursor juga tidak bergeser
`13h`	`AL` = mode tulis; `BH` = halaman; `BL` = atribut; `CX` = panjang string; `DH`, `DL` = posisi; `ES:BP` = alamat string	—	Menulis seluruh string sekaligus; mode `AL` = `01h` memindahkan kursor ke akhir string

Fungsi `09h` menulis satu karakter yang dapat diulang sebanyak nilai `CX` tanpa menggeser kursor, sehingga tepat untuk membuat blok warna. Fungsi `13h` mencetak seluruh string pada `ES:BP`, sehingga `DS` harus disalin ke `ES` lebih dahulu.

1.2.3 Byte Atribut Warna

Warna pada mode teks ditentukan satu byte **atribut** dengan susunan bit yang tetap: bit 3–0 menentukan warna huruf (*foreground*), bit 6–4 menentukan warna latar (*background*), dan bit 7 menentukan apakah huruf berkedip. Nilai `07h` berarti abu-abu terang pada latar hitam (bawaan DOS), `0Eh` kuning, `1Fh` putih pada latar biru, dan `4Eh` kuning pada latar merah. Pemakaian `INT 10h` lebih aman daripada menulis langsung ke memori video pada alamat `B800h`, karena BIOS sudah mengurus alamat karakter

dan atributnya.

1.2.4 `INT 16h`: Layanan Keyboard

Interupsi `16h` menyediakan tiga fungsi utama.

Fungsi `00h` — baca tombol. Prosesor menunggu sampai ada tombol ditekan, lalu mengembalikan `AH` = *scan code* dan `AL` = kode ASCII. Untuk tombol panah, kode ASCII bernilai nol sehingga `AH` yang harus diperiksa.

Fungsi `01h` — periksa status tombol. Fungsi ini tidak menunggu. Bila ada tombol pada penyangga keyboard, `ZF` bernilai nol dan `AX` berisi tombol tersebut tanpa mengeluarkannya.

Fungsi `02h` — status tombol pengubah. Fungsi ini mengembalikan keadaan `Shift`, `Ctrl`, `Alt`, dan `Caps Lock` pada `AL`.

Masukan DOS melakukan *echo* ke layar dan menghormati tombol `Ctrl`+`C`, sedangkan masukan BIOS tidak melakukan keduanya. Program yang memakai BIOS bertanggung jawab mencetak sendiri karakter yang dibaca.

1.2.5 `INT 17h`: Layanan Printer

Interupsi `17h` mengirim satu byte ke printer melalui port paralel: fungsi `00h` mencetak karakter pada `AL`, fungsi `01h` menginisialisasi printer, dan fungsi `02h` membaca statusnya. Nilai balikan pada `AH` berupa bit status, antara lain bit 1 waktu tunggu habis, bit 3 kesalahan masukan/keluaran, bit 5 kertas habis, dan bit 7 printer sibuk.

1.3 Interupsi DOS dan Penangan Buatan Sendiri

1.3.1 Dua Tingkat Layanan

BIOS menyediakan akses ke perangkat keras, tetapi belum mengenal berkas, direktori, atau konsep "program yang sedang berjalan". Semua itu adalah tugas **DOS** (*Disk Operating System*) yang dibangun di atas BIOS melalui interupsi `21h`: BIOS menggerakkan perangkat, DOS mengatur pemakaian sistem secara keseluruhan.

1.3.2 Fungsi `INT 21h` yang Penting

Konvensi pemanggilan `INT 21h` sama dengan konvensi BIOS: `AH` berisi nomor fungsi dan register lain berisi argumen. Fungsi yang dipakai sepanjang bab ini dirangkum pada Tabel 1.4.

Tabel 1.4: Fungsi `INT 21h` yang dipakai pada bab ini.

Fungsi	Register masukan	Register keluaran	Keterangan
`01h`	`AH` = `01h`	`AL` = karakter	Membaca satu karakter dan menampilkannya (*echo*); menunggu sampai ada tombol
`02h`	`AH` = `02h`; `DL` = karakter	—	Mencetak satu karakter pada layar
`09h`	`AH` = `09h`; `DS:DX` = alamat string	—	Mencetak string yang diakhiri tanda `'$'`; tanda itu sendiri tidak dicetak
`0Ah`	`AH` = `0Ah`; `DS:DX` = alamat penyangga	Penyangga terisi	Membaca string yang diketik pengguna sampai tombol `Enter`; karakter ditampilkan (*echo*)
`25h`	`AH` = `25h`; `AL` = nomor vektor; `DS:DX` = alamat penangan	—	Mengganti isi entri vektor dengan alamat penangan baru
`35h`	`AH` = `35h`; `AL` = nomor vektor	`ES:BX` = alamat penangan	Membaca isi entri vektor yang sedang berlaku
`4Ch`	`AH` = `4Ch`; `AL` = kode keluar	—	Mengakhiri program, menutup berkas, dan mengembalikan kendali ke DOS

Catatan

Pasangan `DS:DX` adalah bentuk baku alamat jauh pada 8086 (Bab **??**): `DS` menyimpan nomor segmen dan `DX` menyimpan offset di dalamnya. Karena itu, program `.EXE` yang memakai fungsi `09h` wajib menginisialisasi `DS`.

1.3.3 Penyangga Fungsi `0Ah`

Fungsi `0Ah` berbeda dari fungsi lain karena pengguna juga menyediakan penyangga (*buffer*) yang tata letaknya ditentukan DOS (Kode 1.1).

```
1 .DATA
2     nama    DB 20           ; byte 0: kapasitas maksimum,
3                             ;   termasuk tombol Enter
4             DB 0            ; byte 1: jumlah karakter yang benar-benar
5                             ;   diketik, diisi oleh DOS
6             DB 20 DUP(0)    ; byte 2 dan seterusnya: isi yang diketik
```

Listing 1.1: Tata letak penyangga untuk fungsi `0Ah`.

Byte pertama harus diisi *sebelum* pemanggilan dan menyatakan kapasitas maksimum, sedangkan byte kedua diisi DOS dengan jumlah karakter yang benar-benar diketik. DOS juga menambahkan karakter `0Dh`, tetapi *tidak* menambahkan tanda `'$'`.

Poin terakhir sering menjebak pemula: karena fungsi `09h` berhenti mencetak hanya ketika menemukan tanda `'$'`, program harus menuliskan tanda itu pada posisi `penyangga + 2 + jumlah karakter`, yaitu posisi `0Dh` yang baru saja ditulis DOS.

Kesalahan yang Sering Terjadi

Bila tanda `'$'` tidak dituliskan, fungsi `09h` akan terus mencetak isi memori sesudah penyangga sampai menemukan byte `24h` (kode ASCII `'$'`). Layar pun dipenuhi karakter sampah. Gejala ini hampir selalu berasal dari kesalahan penulisan tanda akhir string, bukan dari kerusakan layanan DOS.

1.3.4 `INT 20h` dan `INT 21h` Fungsi `4Ch`

Dua cara mengakhiri program telah diperkenalkan: `INT 20h` pada Bab **??** dan fungsi `4Ch`. Perbandingannya tampak pada Tabel 1.5.

Tabel 1.5: Perbandingan cara mengakhiri program pada DOS.

Aspek	`INT 20h`	`INT 21h` fungsi `4Ch`
Bentuk program	Cocok untuk `.COM`	Dapat dipakai `.COM` maupun `.EXE`
Prasyarat	`CS` harus menunjuk ke segmen awal program	Tidak ada prasyarat khusus
Kode keluar	Tidak dapat disertakan	Diberikan pada `AL`, dapat dibaca program pemanggil
Pembersihan oleh DOS	Terbatas	Menutup berkas terbuka dan memulihkan vektor `Ctrl+C`
Pemakaian pada praktikum	Jarang, hanya pada contoh `.COM`	Bentuk baku program `.EXE`

Kesimpulannya, fungsi `4Ch` lebih umum dan lebih aman sehingga menjadi pilihan baku; `INT 20h` tetap perlu dikenal karena muncul pada contoh program lama.

1.3.5 Memasang Penangan Interupsi Sendiri

Karena alamat penangan disimpan pada tabel vektor, mengganti isi sebuah entri berarti mengganti seluruh perilaku interupsi tersebut. Prosedurnya ada lima langkah: simpan penangan lama dengan fungsi `35h` (hasilnya pada `ES:BX`), siapkan alamat penangan baru pada `DS:DX`, pasang dengan fungsi `25h`, uji dengan `INT n`, lalu pulihkan penangan lama. Contoh pada Kode 1.2 memakai vektor `60h` yang tidak dipakai perangkat keras mana pun sehingga aman untuk percobaan.

```
    MOV AH, 35h                 ; fungsi 35h: baca vektor
    MOV AL, 60h                 ; nomor vektor 60h
    INT 21h                     ; keluaran: ES:BX = penangan lama
    PUSH ES
    PUSH BX                     ; simpan penangan lama di stack

    PUSH DS                     ; simpan DS program
    MOV DX, OFFSET handler60    ; DX = offset penangan baru
    MOV AX, SEG handler60       ; AX = segmen penangan baru
    MOV DS, AX                  ; DS:DX = alamat penangan baru
    MOV AH, 25h                 ; fungsi 25h: pasang vektor
    MOV AL, 60h
    INT 21h
    POP DS                      ; pulihkan DS program

    INT 60h                     ; uji: jalankan penangan baru
```

Listing 1.2: Menyimpan vektor lama dan memasang penangan baru.

Penangan itu sendiri adalah prosedur jauh yang diakhiri `IRET` (Kode 1.3). Karena dipanggil lewat interupsi dan bukan `CALL`, register yang dipakainya harus disimpan dan dipulihkan sendiri.

```
handler60 PROC FAR
    PUSH AX                     ; simpan register yang akan dipakai
    PUSH DX
    MOV AH, 09h                 ; cetak pesan sapaan
    LEA DX, pesan
    INT 21h
    POP DX                      ; pulihkan register
    POP AX
    IRET                        ; pop IP, CS, lalu FLAGS
handler60 ENDP
```

Listing 1.3: Penangan interupsi `60h` yang diakhiri `IRET`.

Sesudah pengujian, vektor lama dipulihkan dengan mengirimkan kembali kedua nilai yang disimpan di awal program ke fungsi `25h` (Kode 1.4).

```
    POP DX                      ; DX = offset penangan lama
    POP DS                      ; DS = segmen penangan lama
    MOV AH, 25h                 ; pasang kembali penangan lama
    MOV AL, 60h
    INT 21h
```

Listing 1.4: Memulihkan vektor interupsi yang semula.

Kesalahan yang Sering Terjadi

Mengganti isi vektor interupsi berisiko. Tiga kesalahan yang paling sering terjadi: penangan tidak diakhiri `IRET` sehingga prosesor melompat ke alamat acak; vektor lama tidak dipulihkan sehingga program berikutnya menerima penangan yang sudah tidak ada; dan penangan tidak menyimpan register yang dipakainya. Vektor `08h` dan `09h` sebaiknya tidak dijadikan percobaan pertama.

Penangan yang menggantikan interupsi perangkat keras umumnya tidak berdiri sendiri: sesudah pekerjaannya selesai, ia melanjutkan kendali ke penangan lama agar fungsi asli perangkat tetap berjalan. Teknik ini disebut **interupsi berantai** (*interrupt chaining*) dan dilaksanakan dengan melompat ke alamat penangan lama, bukan dengan `IRET`.

1.4 Contoh Program

Dua contoh berikut memperlihatkan dua sisi bab ini: contoh pertama memakai layanan DOS, contoh kedua memakai layanan BIOS.

1.4.1 Contoh 1: Menyapa Pengguna dengan `INT 21h`

Program pada Kode 1.5 meminta nama pengguna, membaca jawabannya dengan fungsi `0Ah`, lalu menyapa pengguna dengan nama tersebut.

```
; ============================================================
; input-nama.asm
; Meminta nama pengguna dengan INT 21h fungsi 0Ah (input string
; berpenyangga), lalu menyapa pengguna dengan nama tersebut.
; Prosesor  : Intel 8086 (mode real, 16-bit)
; Assembler : MASM / TASM / Emu8086
; ============================================================

.MODEL SMALL
```

```
.STACK 100h

.DATA
    tanya   DB  'Siapa nama Anda? $'          ; pertanyaan pembuka
    sapaan  DB  0Dh, 0Ah, 'Halo, $'           ; awal kalimat sapaan
    nama    DB  20                            ; byte 0: kapasitas maksimum
            DB  0                             ; byte 1: jumlah terisi (diisi DOS
    ↪ )
            DB  20 DUP(0)                     ; byte 2..: isi nama yang diketik
    penutup DB  '! Selamat belajar assembly.', 0Dh, 0Ah, '$'

.CODE
START:
    ; ---------- 1. inisialisasi register segmen data ----------
    MOV AX, @DATA
    MOV DS, AX                            ; DS menunjuk segmen .DATA

    ; ---------- 2. cetak pertanyaan ----------
    MOV AH, 09h                           ; fungsi 09h: cetak string '$'
    LEA DX, tanya                         ; DX = offset string pertanyaan
    INT 21h

    ; ---------- 3. baca nama pengguna ----------
    MOV AH, 0Ah                           ; fungsi 0Ah: input string berpenyangga
    LEA DX, nama                          ; DX = offset awal penyangga
    INT 21h                               ; DOS mengisi byte 1 dan byte 2..

    ; ---------- 4. sisipkan tanda '$' sesudah nama ----------
    LEA SI, nama                          ; SI menunjuk awal penyangga
    MOV BL, [SI+1]                        ; BL = jumlah karakter yang diketik
    MOV BH, 0                             ; BH = 0, sehingga BX = jumlah karakter
    ADD SI, 2                             ; SI menunjuk karakter pertama nama
    ADD SI, BX                            ; SI menunjuk sesudah karakter terakhir
    MOV BYTE PTR [SI], '$'                ; ganti 0Dh dengan penanda akhir '$'

    ; ---------- 5. cetak sapaan beserta nama ----------
    MOV AH, 09h
    LEA DX, sapaan                        ; cetak "Halo, "
    INT 21h

    MOV AH, 09h
    LEA DX, nama+2                        ; cetak nama yang diketik pengguna
    INT 21h

    MOV AH, 09h
```

```
    LEA DX, penutup                     ; cetak penutup kalimat
    INT 21h

    ; ---------- 6. akhiri program ----------
    MOV AH, 4Ch                         ; fungsi 4Ch: terminasi program
    MOV AL, 00h                         ; AL = 0, keluar tanpa kesalahan
    INT 21h

END START
```

Listing 1.5: Program `input-nama.asm` meminta nama lalu menyapa pengguna.

Alur program ini terdiri atas enam tahap.

1. **Inisialisasi `DS`.** Karena program berbentuk `.EXE`, `DS` harus diarahkan ke segmen data.
2. **Pertanyaan dan masukan.** Fungsi `09h` mencetak string `tanya`, lalu fungsi `0Ah` membaca jawaban pengguna. DOS menyimpan jumlah karakter pada `nama+1` dan isi nama mulai `nama+2`.
3. **Penyisipan tanda `'$'`.** Instruksi `MOV BL, [SI+1]` mengambil jumlah karakter, lalu `ADD SI, BX` menggeser `SI` ke posisi sesudah karakter terakhir. `0Dh` yang ditulis DOS pada posisi itu digantikan tanda `'$'`.
4. **Pencetakan sapaan dan terminasi.** Tiga pemanggilan fungsi `09h` mencetak kata "Halo, ", isi penyangga, dan kalimat penutup; fungsi `4Ch` mengakhiri program.

```
Siapa nama Anda? Andi
Halo, Andi! Selamat belajar assembly.
```

Listing 1.6: Contoh keluaran program `input-nama.asm`.

Program tidak perlu mencetak sendiri karakter yang diketik pengguna: fungsi `0Ah` sudah menampilkannya (*echo*) sambil menyimpannya ke penyangga.

1.4.2 Contoh 2: Teks Berwarna dengan `INT 10h`

Program pada Kode 1.7 menampilkan judul berwarna kuning dan sebuah garis merah dengan layanan video BIOS. Tidak satu pun layanan DOS dipakai untuk mencetak.

```
; ============================================================
; warna-layar.asm
; Menampilkan teks berwarna dengan layanan video BIOS:
;   INT 10h fungsi 00h - menetapkan mode teks 80x25 berwarna
```

```
;   INT 10h fungsi 13h - menulis string beserta atribut warnanya
;   INT 10h fungsi 02h - memindahkan posisi kursor
;   INT 10h fungsi 09h - menulis satu karakter berulang
; Prosesor  : Intel 8086 (mode real, 16-bit)
; Assembler : MASM / TASM / Emu8086
; ============================================================

.MODEL SMALL
.STACK 100h

.DATA
    judul   DB  'Bahasa Assembly - Interupsi BIOS/DOS'
    panjang EQU $ - judul                       ; panjang judul = 36 karakter
    garis   DB  '===================================='   ; 36 tanda '='

.CODE
START:
    ; ---------- 1. inisialisasi register segmen ----------
    MOV AX, @DATA
    MOV DS, AX                          ; DS menunjuk segmen .DATA
    MOV ES, AX                          ; ES = DS, untuk alamat ES:BP

    ; ---------- 2. tetapkan mode video 03h ----------
    MOV AH, 00h                         ; fungsi 00h: set mode video
    MOV AL, 03h                         ; 03h = teks 80x25 berwarna
    INT 10h                             ; layar dibersihkan dan diatur

    ; ---------- 3. tulis judul berwarna pada baris 5 ----------
    MOV AH, 13h                         ; fungsi 13h: tulis string
    MOV AL, 01h                         ; mode 01h: kursor maju sesudah menulis
    MOV BH, 00h                         ; halaman tampilan 0
    MOV BL, 0Eh                         ; atribut 0Eh = kuning pada hitam
    MOV CX, panjang                     ; CX = jumlah karakter yang ditulis
    MOV DH, 05h                         ; DH = baris 5
    MOV DL, 0Ah                         ; DL = kolom 10
    LEA BP, judul                       ; BP = offset string judul
    INT 10h                             ; ES:BP menunjuk string yang ditulis

    ; ---------- 4. pindahkan kursor ke baris 7 ----------
    MOV AH, 02h                         ; fungsi 02h: set posisi kursor
    MOV BH, 00h                         ; halaman 0
    MOV DH, 07h                         ; baris 7
    MOV DL, 0Ah                         ; kolom 10
    INT 10h

```

```
    ; ---------- 5. tulis garis '=' berwarna merah ----------
    MOV AH, 09h                         ; fungsi 09h: tulis karakter berulang
    MOV AL, '='                         ; AL = karakter yang ditulis
    MOV BH, 00h                         ; halaman 0
    MOV BL, 0Ch                         ; atribut 0Ch = merah terang pada hitam
    MOV CX, panjang                     ; tulis sebanyak panjang judul (35)
    INT 10h                             ; fungsi 09h tidak menggeser kursor

    ; ---------- 6. kembalikan mode teks lalu keluar ----------
    MOV AH, 4Ch                         ; fungsi 4Ch: terminasi program
    MOV AL, 00h
    INT 21h

END START
```

Listing 1.7: Program `warna-layar.asm` menampilkan teks berwarna dengan layanan video BIOS.

Hal-hal penting pada program ini adalah sebagai berikut.

- `MOV ES, AX` menyalin segmen data ke `ES`, karena fungsi `13h` membaca alamat string dari `ES:BP`.
- Fungsi `13h` dengan `AL` = `01h` menulis seluruh string pada posisi `DH`, `DL` dan memindahkan kursor ke akhir string. Warna ditentukan `BL` = `0Eh`, yaitu `00001110b`: bit 3–0 bernilai `Eh` (kuning terang) dan bit 6–4 bernilai `0h` (latar hitam).
- Fungsi `02h` memindahkan kursor ke baris 7 tanpa mencetak apa pun, sehingga garis merah tercetak di bawah judul.
- Fungsi `09h` menulis karakter `'='` sebanyak `CX` kali dengan satu pemanggilan. Karena kursor tidak bergeser, garis terbentuk pada posisi yang disiapkan fungsi `02h`.

```
        Bahasa Assembly - Interupsi BIOS/DOS
        ====================================
```

Listing 1.8: Gambaran keluaran program `warna-layar.asm` pada layar teks. Judul tercetak kuning dan garis tercetak merah.

Studi Kasus

Studi kasus 1: sapaan pribadi. Program `input-nama.asm` adalah pola baku setiap program dialog: cetak pertanyaan, baca masukan, cetak tanggapan. Pola ini dipakai ulang pada Tugas 7.

Studi kasus 2: tampilan berwarna. Karena yang memindahkan kursor pada `warna-layar.asm` adalah BIOS, DOS tidak mengetahui posisi kursor yang sesungguhnya. Bila program kembali memakai fungsi `09h`, keluarannya muncul pada posisi yang dilacak DOS. Kedua cara pencetakan sebaiknya tidak dicampur tanpa mengatur ulang kursor dengan fungsi `02h`.

1.5 Praktikum

Praktikum

Judul: Operasi masukan/keluaran dengan `INT 21h` dan penelusuran stack pada pemanggilan `INT`.

Sub-CPMK: 2.4 • **Alokasi waktu:** 100 menit • **Bentuk:** individu • **Tugas 7 dikumpulkan pada praktikum ini.**

1.5.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu menulis program masukan/keluaran dengan `INT 21h` dan menunjukkan isi stack sebelum serta sesudah instruksi `INT` dijalankan.

1.5.2 Alat dan Bahan

- Komputer dengan Emu8086, GUI Turbo Assembler, atau DOSBox beserta DEBUG.
- Berkas `input-nama.asm` dan `warna-layar.asm` dari direktori `code/asm/`.
- Jurnal praktikum untuk mencatat nilai register dan isi stack.

1.5.3 Langkah Kerja

A. Masukan/keluaran dengan `INT 21h`

1. Rakit dan jalankan `input-nama.asm` seperti pada Kode 1.5, lalu catat keluarannya.
2. Ubah kapasitas penyangga dari `20` menjadi `5`, lalu ketik nama yang lebih panjang dari empat karakter dan jelaskan hasilnya dengan mengacu pada Kode 1.1.
3. Hapus baris `MOV BYTE PTR [SI], '$'`, rakit ulang, dan jalankan. Amati keluarannya dan jelaskan penyebabnya.
4. Tambahkan satu baris keluaran yang mencetak NIM mahasiswa memakai fungsi `09h`.

B. Menelusuri Stack pada Pemanggilan `INT`

1. Buka DOSBox, lakukan `mount` ke direktori kerja, lalu rakit program dengan `tasm` dan `tlink`.
2. Jalankan `debug input-nama.exe` dan tampilkan instruksi program dengan perintah `U`. Tandai alamat instruksi `INT 21h` yang memanggil fungsi `09h`.
3. Tampilkan register dengan perintah `R`, lalu catat nilai `SP`, `CS`, `IP`, dan `FLAGS`.
4. Jalankan `T` sampai tepat sebelum `INT 21h`, catat nilai `SP`, lalu tampilkan puncak stack dengan `D SS:SP L 6` dan catat enam byte yang tampak.
5. Jalankan `T` satu kali untuk mengeksekusi `INT 21h`, lalu ulangi perintah `D SS:SP L 6`.
6. Bandingkan hasil langkah 4 dan 5, jelaskan mengapa `SP` berubah, dan tentukan nilai mana yang merupakan `IP`, `CS`, serta `FLAGS`.
7. Ulangi langkah 4 dan 5 dengan `P` sebagai pembanding, lalu bandingkan hasilnya dengan jendela `Stack` pada Emu8086.

Catatan

Perintah `T` (*trace*) menjalankan satu instruksi dan *masuk* ke rutin penangan, sedangkan `P` (*proceed*) menjalankannya sampai selesai tanpa masuk ke penangan. Keduanya memanfaatkan `TF` seperti dijelaskan pada Subbab 1.1.

C. Tugas 7: Sapaan Berwarna

Tugas 7 menggabungkan kedua contoh program pada bab ini. Buatlah program baru bernama `tugas7.asm` dengan ketentuan berikut.

1. Program meminta nama pengguna dengan `INT 21h` fungsi `0Ah` dan kapasitas penyangga `30` karakter, lalu meminta NIM pada penyangga terpisah.
2. Program menetapkan mode video `03h` dengan `INT 10h` fungsi `00h`, lalu mencetak baris `Halo, <nama>` dengan fungsi `13h` dan atribut kuning.
3. Program mencetak baris `NIM: <nim>` dengan atribut hijau terang, lalu menutup tampilan dengan garis merah seperti pada `warna-layar.asm`.
4. Program diakhiri `INT 21h` fungsi `4Ch` dan memuat komentar berbahasa Indonesia pada setiap baris penting.

1.5.4 Hasil yang Dikumpulkan

1. Berkas `tugas7.asm` beserta hasil perakitan dan tangkapan layar keluarannya.

2. Tangkapan layar keluaran program pada langkah A.3 beserta penjelasan penyebabnya.
3. Tabel pengamatan nilai `SP` dan isi puncak stack sebelum serta sesudah `INT 21h`, beserta jawaban langkah B.6 dan B.7.

Asesmen (Evaluasi Kinerja)

Asesmen praktikum Bab 1 (bobot 10% dari komponen praktikum).

Indikator	Bobot	Kriteria
Program masukan/keluaran dengan `INT 21h` berjalan benar	20%	Ketiga fungsi (`0Ah`, `09h`, `02h`) dipakai dengan argumen yang tepat
Tugas 7 selesai sesuai ketentuan	35%	Seluruh ketentuan terpenuhi dan teks berwarna tampil benar
Ketepatan tabel pengamatan stack	25%	Nilai `SP` dan isi stack tercatat benar pada setiap langkah
Penjelasan hasil pengamatan	10%	Alasan perubahan `SP` dan makna `T` serta `P` dijelaskan dengan tepat
Kerapian kode dan jurnal praktikum	10%	Komentar lengkap dan jurnal dapat diulang orang lain

1.6 Latihan

1. **Konseptual.** Jelaskan mengapa mekanisme interupsi lebih efisien daripada *polling*. Sertakan satu contoh perangkat pada komputer yang lebih baik diperiksa dengan interupsi.
2. **Konseptual.** Sebutkan tiga jenis interupsi pada prosesor 8086. Untuk setiap jenis, berikan satu contoh beserta nomor vektornya dan siapa yang memicunya.
3. **Perhitungan vektor.** Dengan memakai Persamaan 1.1, tentukan alamat entri vektor `0Ah`, `13h`, dan `21h`. Sebutkan pula alamat byte pertama (offset) dan byte ketiga (segmen) setiap entri.
4. **Analisis stack.** Sebuah program sedang berjalan dengan `SP` = `FF00h` ketika

instruksi `INT 21h` dijalankan.

(a) Berapa nilai `SP` sesudah ketiga penyimpanan pada mekanisme `INT` selesai?

(b) Nilai apa yang berada pada `SS:SP`, `SS:SP+2`, dan `SS:SP+4`?

(c) Mengapa `RET` tidak dapat menggantikan `IRET` pada rutin penangan interupsi?

5. **Analisis BIOS.** Perhatikan dua potongan program berikut.

```
; potongan A                    ; potongan B
MOV AH, 09h                    MOV AH, 13h
MOV AL, '*'
MOV BH, 00h
MOV BL, 0Ah
MOV CX, 10
INT 10h                        INT 10h
```

(a) Apa yang tercetak oleh potongan A dan pada warna apa karakter itu tampil?

(b) Bagian mana pada potongan B yang belum lengkap?

(c) Jelaskan satu perbedaan perilaku kursor antara fungsi `09h` dan `13h`.

6. **Analisis atribut warna.** Tentukan warna huruf dan warna latar untuk atribut `1Eh`, `4Fh`, dan `07h`, dengan menyebut nilai bit 3–0, bit 6–4, dan bit 7.

7. **Penulisan program.** Tulislah program lengkap yang menampilkan kata `SELAMAT` pada baris 10, kolom 20, dengan warna hijau terang pada latar hitam. Gunakan `INT 10h` fungsi `02h` untuk menempatkan kursor dan fungsi `13h` untuk menulis string. Sertakan inisialisasi register segmen yang diperlukan.

8. **Analisis kesalahan.** Seorang mahasiswa memasang penangan interupsi pada vektor `60h` dengan langkah seperti pada Kode 1.2, tetapi program selalu membeku ketika `INT 60h` dijalankan. Setelah diperiksa, rutin penangan yang ditulis diakhiri `RET` dan vektor lama tidak pernah dipulihkan.

(a) Mengapa `RET` menyebabkan program membeku?

(b) Apa akibatnya bila vektor lama tidak dipulihkan?

(c) Tuliskan perbaikan yang diperlukan.

Refleksi

Jawab pertanyaan berikut pada jurnal praktikum Anda.

1. Setelah melihat isi stack secara langsung pada penelusuran DEBUG, apakah gambaran Anda tentang cara kerja `INT` berubah? Bagian mana yang paling mengejutkan?

2. Konsep mana pada bab ini yang masih terasa kabur, dan rencana belajar apa yang akan Anda lakukan untuk mengatasinya?

1.7 Rangkuman

Pada bab ini telah dibahas:

- Interupsi memaksa prosesor menghentikan sementara alur program untuk melayani suatu kejadian, lalu melanjutkannya kembali. Mekanisme ini lebih hemat daripada *polling*.
- Berdasarkan sumbernya, interupsi dibagi menjadi interupsi perangkat keras (`INTR` dan `NMI`), interupsi perangkat lunak (`INT n`), dan pengecualian yang dibangkitkan prosesor, misalnya pembagian dengan nol.
- Tabel vektor interupsi menempati alamat `0000h` sampai `03FFh` pada segmen nol. Setiap vektor berukuran 4 byte (2 byte offset dan 2 byte segmen penangan), sehingga alamat entri vektor ke-n adalah $n \times 4$.
- Instruksi `INT n` menyimpan `FLAGS`, `CS`, lalu `IP` ke stack, mengosongkan `IF` dan `TF`, kemudian memuat `CS:IP` dari entri vektor. Instruksi `IRET` mengembalikan ketiganya dalam urutan terbalik, setara `RET` ditambah `POPF`.
- BIOS melayani perangkat keras (`INT 10h` video, `INT 16h` keyboard, `INT 17h` printer), sedangkan `INT 21h` melayani keperluan tingkat program: masukan (`01h`, `0Ah`), keluaran (`02h`, `09h`), pengelolaan vektor (`25h`, `35h`), dan terminasi (`4Ch`).
- Fungsi `0Ah` memakai penyangga yang tata letaknya ditentukan DOS: byte 0 berisi kapasitas, byte 1 diisi jumlah karakter, dan byte 2 dan seterusnya berisi karakter yang diketik. Tanda `'$'` harus disisipkan program bila isi penyangga akan dicetak dengan fungsi `09h`.
- Isi vektor interupsi dapat diganti melalui fungsi `25h`. Penangan baru wajib menyimpan register yang dipakainya dan diakhiri `IRET`, sedangkan vektor lama wajib dipulihkan.

Checklist Pencapaian Kompetensi

Tandai setiap butir yang sudah Anda kuasai.

- ❑ Saya dapat menjelaskan alasan diperlukannya interupsi dan membedakan ketiga jenisnya.
- ❑ Saya dapat menghitung alamat entri vektor ke-n dan menyebutkan vektor penting beserta kegunaannya.

- ❑ Saya dapat menuliskan isi stack sesudah `INT` dijalankan beserta urutan pengambilan oleh `IRET`.
- ❑ Saya dapat menjelaskan hubungan `TF` dengan fasilitas eksekusi satu langkah pada *debugger*.
- ❑ Saya dapat memakai `INT 10h` dan `INT 21h` untuk menampilkan teks berwarna serta membaca masukan pengguna.
- ❑ Saya dapat menjelaskan perbedaan `INT 20h` dan `INT 21h` fungsi `4Ch`.
- ❑ Saya dapat memasang dan memulihkan sebuah vektor interupsi dengan aman.

Bila masih ada butir yang belum bertanda, ulangi bagian yang bersesuaian.

Catatan

Rujukan bab. Daftar lengkap vektor interupsi beserta setiap fungsinya dapat ditelusuri pada [**?**]. Penjelasan resmi mekanisme interupsi pada prosesor 8086 terdapat pada [**?**], sedangkan contoh penggunaan *debugger* untuk menelusuri interupsi dibahas pada [**?**]. Keterangan praktis mengenai emulator yang dipakai pada praktikum tersedia pada [**?**].

1.8 Evaluasi

1.8.1 Soal Pilihan Ganda

1. Setiap entri pada tabel vektor interupsi 8086 berukuran ...
   (a) 2 byte berisi offset penangan
   (b) 4 byte: 2 byte offset dan 2 byte segmen
   (c) 4 byte berisi nomor vektor
   (d) 8 byte: `IP`, `CS`, dan `FLAGS`
2. Alamat entri untuk vektor `10h` adalah ...
   (a) `0010h`
   (b) `0040h`
   (c) `0100h`
   (d) `0160h`
3. Urutan penyimpanan ke stack ketika instruksi `INT n` dijalankan adalah ...
   (a) `IP`, `CS`, `FLAGS`

(b) `CS`, `IP`, `FLAGS`

(c) `FLAGS`, `CS`, `IP`

(d) `FLAGS`, `IP`, `CS`

4. Instruksi yang wajib mengakhiri sebuah rutin penangan interupsi adalah ...

   (a) `RET`

   (b) `IRET`

   (c) `HLT`

   (d) `POPF`

5. Fungsi `INT 21h` yang dipakai untuk membaca string yang diketik pengguna beserta penyangganya adalah ...

   (a) `01h`

   (b) `02h`

   (c) `09h`

   (d) `0Ah`

1.8.2 Soal Uraian

1. Jelaskan bagaimana prosesor menemukan rutin penangan ketika instruksi `INT 16h` dijalankan, lengkap dengan perhitungan alamat entri vektornya.
2. Bandingkan `INT 20h` dengan `INT 21h` fungsi `4Ch`. Sebutkan tiga perbedaan dan jelaskan kapan masing-masing cara dipakai.
3. Jelaskan tata letak penyangga fungsi `0Ah` dan tuliskan potongan program yang menyisipkan tanda `'$'` sesudah karakter terakhir. Jelaskan pula akibatnya bila tanda tersebut tidak disisipkan.
4. Tuliskan program lengkap yang meminta nama pengguna dengan `INT 21h` fungsi `0Ah`, lalu menampilkan `Halo, <nama>` pada baris 12, kolom 15 dengan `INT 10h` fungsi `13h` dan atribut kuning pada latar hitam. Sertakan inisialisasi register segmen dan penulisan tanda `'$'` pada penyangga.

1.8.3 Kisi-kisi Penilaian

Tabel 1.6: Kisi-kisi evaluasi Bab 1.

Sub-CPMK	Indikator yang diukur	Bentuk soal	Bobot
2.4	Menjelaskan konsep interupsi dan tabel vektor beserta perhitungan alamatnya	Pilihan ganda 1, 2; Uraian 1	25%
2.4	Menjelaskan mekanisme `INT` dan `IRET` pada stack	Pilihan ganda 3, 4; Uraian 1	25%
2.4	Menganalisis layanan BIOS dan DOS beserta penyangganya	Pilihan ganda 5; Uraian 2, 3	25%
2.4	Menulis program masukan/keluaran dengan `INT 21h` dan `INT 10h`	Uraian 4	25%

Checklist Pencapaian Kompetensi

Checklist Pencapaian Kompetensi Bab 1

Seorang mahasiswa dinyatakan kompeten pada Sub-CPMK 2.4 apabila:

- ✓ dapat menjelaskan alasan diperlukannya interupsi dan membedakan interupsi perangkat keras, perangkat lunak, serta pengecualian;
- ✓ dapat menghitung alamat entri vektor ke-n dan menyebutkan vektor-vektor penting beserta kegunaannya;
- ✓ dapat menuliskan isi stack sesudah `INT` dijalankan dan menjelaskan urutan pengambilan oleh `IRET`;
- ✓ dapat memakai layanan `INT 10h`, `INT 16h`, dan `INT 21h` dengan argumen register yang benar;
- ✓ dapat menulis program masukan/keluaran lengkap beserta penanganan penyangga fungsi `0Ah`;
- ✓ dapat memasang sebuah penangan interupsi sederhana dan memulihkan vektor semula;
- ✓ dapat menjelaskan risiko penggantian vektor dan cara menghindarinya.